

今月のフロントエンド

フロントエンド エンジニア集会 2026 年 8 月

"今月のフロントエンド" とは

今月あったフロントエンドのニュースを, 以下のフォーマットで紹介します.

- ニュースがあった技術について, その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何があったかを解説

取り上げないもの

- 特定フレームワークに関するバージョンアップ (例外あり)
- AI 単品のニュース

Oxlint

2026/07/22

"Oxlint" とは

- **Rust 製** の JavaScript / TypeScript 向け Linter (コードの書き方を検査するツール)
- 定番の ESLint と同じ役割を, 桁違いの速さで実行することを目指している
- Vite を開発する VoidZero (2026 年 6 月に Cloudflare が買収) が開発している

Oxlint のニュース

型情報を使った Lint が正式版に — ESLint 比で最大 18 倍

- 7/22, 型を見る Lint (type-aware) が実験版から **安定版** へ
- typescript-eslint が持つ型ありルール **61 個のうち 59 個** をカバー
- VS Code 規模のコードベースで **ESLint + typescript-eslint の最大 18 倍** 速い
- 設定は `typeAware: true` の 1 行

なぜこれが大きいのか

- Lint には **書き方だけ見るもの** と **型情報まで見るもの** の 2 種類がある
 - 「Promise の `await` 忘れ」のようなバグは, 後者でないと見つけられない
- 型ありルールは重く, ESLint では CI で数分かかることも珍しくなかった
- Oxlint は速さが売りだったが, **型ありルールが無い** ことが最大の弱点だった
 - 「速いが物足りない」から「置き換え候補」へ

TypeScript 7 との関係

- 型を見る部分は **tsgolint** という別エンジンが担当している
 - tsgolint は **Go 製の TypeScript 7.0 コンパイラの上に直接構築**
- Lint と型チェックが **同じ型情報を共有** するため, 二重に解析しなくて済む

Google Chrome

2026/07/28



"Google Chrome" とは

- Google が開発する, デスクトップで圧倒的シェアを持つ Web ブラウザ
- レンダリングエンジンは Blink, 新しい CSS / Web API の実装を積極的に主導
- Edge など他の Chromium 系ブラウザの基盤にもなっている

Google Chrome のニュース

Chrome 151 — SPA の "画面遷移" がようやく計測できるようになった

- `soft-navigation` : JavaScript による **画面内の遷移** を 1 回の遷移として記録するエントリが追加された
- 遷移のたびに **計測の起点がリセット** され, 以降の数値は「いま表示している画面のもの」として集計される
- 併せて `interaction-contentful-paint` (操作後に新しい内容が描画されるまで) も追加

従来の計測 API と何が違うのか

- これまでの計測は Navigation Timing API
 - ページを読み込み直す遷移 (ハードナビゲーション) のみを計測する
- SPA の画面切り替えは JavaScript が DOM を書き換える動作をする
 - この API から見ると 「何も起きていない」

何が嬉しいのか

- Core Web Vitals (Google が定める体感速度の指標) を **画面ごとに** 出せる
→ 「どの画面が遅いのか」が, ようやく数字で分かる
- React / Next.js のような SPA ほど効果が大い
→ これまで見えていなかった遷移が, まとめて可視化される
- `interaction-contentful-paint` は「押したのに何も出ない」時間を捉えられる

他ブラウザとの互換性

機能	Chrome / Edge	Safari	Firefox
soft-navigation	151～	—	—
interaction-contentful-paint	151～	—	—

- 現状は **Chromium 系のみ**. Safari / Firefox は実装の表明もない
- 計測系なので **非対応でも表示は変わらない** → 段階的に導入できる
- ただし数字が取れるのは Chrome 利用者の分だけ — **母数が偏る点には注意**

Cloudflare

2026/08/06

"Cloudflare" とは

- 世界中の Web サイトが利用する CDN (配信を高速化する仕組み) / セキュリティ事業者
- 利用者に近いサーバでコードを動かす実行基盤 **Workers** を提供している
- 2026 年 6 月に Vite の開発元 VoidZero を買収し, フロントエンド基盤側にも進出した

Cloudflare のニュース

"Kitesurf" — Chromium を使わない ヘッドレスブラウザが登場

- 8/6 公開. **Chromium** を一切使わず, Workers の V8 隔離環境だけで動く
- HTML の処理には Rust 製の **Blitz**, CSS エンジンには Firefox の **Styleo** を採用
- **Chrome DevTools Protocol 互換** — Puppeteer / Playwright がそのまま繋がる
- スクリーンショット 1 枚あたり CPU **1,173ms → 380ms**, メモリ **271MiB → 58MiB**

完全に AI 向けのブラウザ

- Cloudflare 自身が **ヘッドレスブラウザ** (画面を持たないブラウザ) と位置づけている
→ 表示用の画面は無いが **描画はする**. スクリーンショットや PDF を出力できる
- 想定用途は **AI エージェント**. ただし E2E テストや OGP 画像生成で使う
ヘッドレスブラウザの置き換え候補 にもなる
- この用途は実質 Chromium 一択で, 1 プロセスあたりの資源消費が重いのが定番の悩みだった

AI 向けブラウザが生まれるということ

- 従来のウェブサイトは "人間向け" と "クローラー向け" の設計をしてきた
- そこに最近, "AI クローラー向け" が急に入ってきている状況だった
- これからは "AI エージェント向け・AI ブラウザ向け" の設計が本格的に必要なになってくるのではないか

JavaScript

2026/08/13



"JavaScript" とは

- Web ブラウザ上で動作する, フロントエンド開発の中心となるプログラミング言語
- 言語仕様は ECMAScript として標準化され, 毎年 1 回改訂される
- 実装はブラウザごとに異なり, Chrome は V8, Firefox は SpiderMonkey, Safari は JavaScriptCore を使う

JavaScript のニュース

`using` 構文が, 3 大エンジンすべてで動くようになった

- 8/13 公開の **Safari 先行版 (Safari Technology Preview 250)** が Explicit Resource Management (明示的なリソース管理) に対応
- `using` / `await using`, `Symbol.dispose`, `DisposableStack` を含む完全対応
- これで **V8 / SpiderMonkey / JavaScriptCore** の 3 エンジンが出揃った
- TypeScript で書いて変換していた構文が, **変換なしで動く** 段階に入る

"using" とは何か

- ブロックを抜けるときに **後片付けを自動で呼ぶ** 変数宣言
 - ファイル, DB 接続, ロックなど「開いたら必ず閉じる」対象が想定用途
- `try { ... } finally { close() }` の書き忘れを, 構文レベルで防げる
- 複数ある場合は **宣言と逆順** に片付けられる. 非同期版が `await using`
- Java の try-with-resources, C# の `using`, Python の `with` に相当する機能

これまでの書き方

```
const file = openFile("data.txt");  
try {  
    read(file);  
} finally {  
    file.close(); // 忘れたら開きっぱなし  
}
```


これからの書き方

```
// 後片付けの手順を、オブジェクト側に定義しておく
const openFile = (path) => ({
  path,
  [Symbol.dispose]() { console.log(`closed: ${path}`); },
});

{
  using file = openFile("data.txt");
  read(file);
} // ← 抜けた瞬間に Symbol.dispose() が呼ばれる
```

複数開くと, 差はもっと開く

// これまで: 1 つ増えるごとに, ネストが深くなる

```
const a = openFile("a.txt");  
try {  
    const b = openFile("b.txt");  
    try { merge(a, b); } finally { b.close(); }  
} finally { a.close(); }
```

// これから: 並べて書くだけ. 片付けは宣言と逆順 ($b \rightarrow a$)

```
using a = openFile("a.txt");  
using b = openFile("b.txt");  
merge(a, b);
```

使えるようになるのはいつか

- Safari は **先行版での対応** — 製品版 (通常の Safari) への搭載はこれから
- Chrome / Firefox は既に安定版で利用可能
- 当面は TypeScript による変換を併用するのが安全
→ 「変換なしで書ける」と言い切れるのは, Safari 製品版に載ってから

おわり

付録: Biome VS Oxlint

同じ Rust 製の Linter, なぜ速度差がつくのか

"Biome" とは

- **Rust 製** の, Linter と Formatter (整形ツール) を兼ねたツール
- ESLint (検査) と Prettier (整形) を **まとめて 1 つで置き換える** ことを目指す
- 2025 年に v2 へ. Oxlint と並ぶ「脱 ESLint」の有力候補として比較されることが多い

問題: どちらが速いか

どちらも Rust 製で, ネイティブバイナリとして動く Linter です

- 片方 (Oxlint) は 検査に専念, もう片方 (Biome) は 整形も兼ねる
- どちらも ESLint とは桁が違う速さ, という点では同じ

どちらが早いでしょうか・・・?

答え: Oxlint が 2～5 倍速い

- oxc 公式のベンチマーク (VS Code のコードベース) では **Oxlint が 2.5～3.3 倍**
- 実プロダクト (TypeScript / TSX 約 800 ファイル) での計測では **約 4.9 倍**
- ただし **どちらも数秒以内** で終わる — ESLint の数十秒とは別の土俵にいる

速さの違いは, 目的の違い

	Oxlint	Biome
役割	Lint のみ	Lint + 整形の 統合
言語	JS / TS	JS / TS, HTML, CSS, JSON など

- Biome は **Web 系の言語を一括で引き受ける** ツールを目指している
→ Vue / Svelte / Astro のファイルも 2.3 から扱える
- Oxlint は **JS / TS の検査に用途を限定** している
→ この目的の違いが, そのままアーキテクチャの違いになっている

目的の違いは、構文木の作りになる

	Oxlint (oxc)	Biome
構文木	AST (抽象構文木)	CST (具象構文木)
空白・コメント	保持しない	すべて保持する
構文エラー	解析を止める	解析を続けられる

- Biome のパーサは**エラー耐性**を持ち, 空白やコメントを含めて **元のソースを 1 文字も失わない木 (CST)** を組み立てる
- Oxlint の木は **検査に必要な情報だけ** — 木そのものが小さい

速さは「割り切り」の結果

- Biome は **整形もでき, 壊れたコードも扱える構造体** を作っている
→ 木が大きく, ノード単位にメモリを確保する分だけ重くなる
- Oxlint の木は **一括確保したメモリ (アリーナ) 上に並ぶ**
→ メモリへのアクセスが一直線になり, oxc はこれだけで約 20% 速くなったと報告
- つまり速度差は最適化の巧拙ではなく, **木に持たせた役割の差** から来ている

型情報の扱いも、思想が違う

	Oxlint	Biome
型の取得	TypeScript 7 に任せる	自前で推論
TypeScript	必要	不要
精度	tsc と同じ	一部のみ

- Biome は「TypeScript 無しで型ありチェック」という別解に賭けている
→ `await` 忘れの検出精度は typescript-eslint の 75～85% 程度
- Oxlint は逆に、型の解釈を 本家に任せて速く回す 方針

自作ルール (プラグイン) の作り方も違う

	Oxlint	Biome
書き方	JavaScript / TS	GritQL (専用言語)
ESLint 資産	そのまま動く	動かない
配布・共有	できる	プロジェクト内のみ

- Oxlint は **ESLint 互換のプラグイン API** (2026/03 アルファ公開)
 - JS で書く以上 **Rust ⇄ JS の往復** は発生する
 - oxc はその往復を **ほぼゼロコスト** にする仕組みで潰しにいった
- Biome は CST への **パターンマッチ**. 独特だが表現力は高い

で, どちらを使うべきか

- 速さ・ルールの網羅性・ESLint 資産の引き継ぎ なら Oxlint
- 整形も含めて 1 つに寄せたい なら Biome
→ 設定ファイル 1 つで完結. JSON / CSS など JS 以外のファイルも扱える
- どちらも ESLint とは桁が違う速さ — **まず ESLint から離れること**の方が本題